

Converting Words to Numbers

What you need is a system for turning a word into an appropriate index number. To begin, you know that computers use various schemes for representing individual characters as numbers. One such scheme is the ASCII code, in which *a* is 97, *b* is 98, and so on, up to 122 for *z*.

The extended ASCII code runs from 0 to 255, to accommodate capitals, punctuation, accents, symbols, and so on. There are only 26 letters in English words, so let's devise our own code, a simpler one that can potentially save memory space. Let's say *a* is 1, *b* is 2, *c* is 3, and so on up to 26 for *z*. We'll also say a blank—the space character—is 0, so we have 27 characters. (Uppercase letters, digits, punctuation, and other characters aren't used in this dictionary.)

How could we combine the digits from individual letter codes into a number that represents an entire word? There are all sorts of approaches. We'll look at two representative ones, and their advantages and disadvantages.

Adding the Digits

A simple approach to converting a word to a number might be to simply add the code numbers for each character. Say you want to convert the word *elf* to a number. First, you convert the characters to digits using our homemade code:

$$e = 5 \quad l = 12 \quad f = 6$$

Then you add them:

$$5 + 12 + 6 = 23$$

Thus, in your dictionary the word *elf* would be stored in the array cell with index 23. All the other English words would likewise be assigned an array index calculated by this process.

How well would this approach work? For the sake of argument, let's restrict ourselves to 10-letter words. Then (remembering that a blank is 0), the first word in the dictionary, *a*, would be coded by

$$0 + 0 + 0 + 0 + 0 + 0 + 0 + 0 + 0 + 1 = 1$$

The last potential word in the dictionary would be *zzzzzzzzzz* (10 letter *z*'s). The code obtained by adding its letters would be

$$26 + 26 + 26 + 26 + 26 + 26 + 26 + 26 + 26 + 26 = 260$$

Thus, the total range of word codes is from 1 to 260 (assuming a string of all spaces is not a word). Unfortunately, there are 50,000 words in the dictionary, so there aren't enough index numbers to go around. If each array element could hold about 192 words (50,000 divided by 260), then you might be able fit them all in, but how would you distinguish among the 192 words in one array element?

Clearly, this coding presents problems if you're thinking in terms of the one word-per-array element scheme. Maybe you could put a subarray or linked list of words at each array element. Unfortunately, such an approach would seriously degrade the access speed. Accessing the array element would be quick, but searching through the 192 words to find the one you wanted could be very slow.

So this first attempt at converting words to numbers leaves something to be desired. Too many words have the same index. Certainly, any anagram of a word would have the same code because the order of the letters doesn't change the value. In addition, these words

acne ago aim baked cable hack

and dozens of other words have letters that add to 23, as *elf* does. For words with higher codes, there could be hundreds of other matching words. It is now clear that this approach doesn't discriminate enough, so the resulting array has too few elements. We need to spread out the range of possible indices.

Multiplying by Powers

Let's try a different way to map words to numbers. If the array was too small before, make sure it's big enough. What would happen if you created an array in which every word—in fact, every potential word, from *a* to *zzzzzzzzzz*—was guaranteed to occupy its own unique array element?

To do this, you need to be sure that every character in a word contributes in a unique way to the final number.

You can begin by thinking about an analogous situation with numbers instead of words. Recall that in an ordinary multidigit number, each digit-position represents a value 10 times as big as the position to its right. Thus 7,546 really means

$$7*1,000 + 5*100 + 4*10 + 6*1$$

Or, writing the multipliers as powers of 10:

$$7*10^3 + 5*10^2 + 4*10^1 + 6*10^0$$

In this system you break a number into its digits, multiply them by appropriate powers of 10 (because there are 10 possible digits), and add the products. If this happened to be an octal number using the digits from 0 to 7, then you would get $7*8^3 + 5*8^2 + 4*8^1 + 6*8^0$.

In a similar way, you can decompose a word into its letters, convert the letters to their numerical equivalents, multiply them by appropriate powers of 27 (because there are 27 possible characters, including the blank), and add the results. This approach gives a unique number for every word.

Let's return to the example of converting the word *elf* to a number. You convert the digits to numbers as shown earlier. Then you multiply each number by the appropriate power of 27 and add the results:

$$5*27^2 + 12*27^1 + 6*27^0$$

Calculating the powers gives

$$5*729 + 12*27 + 6*1$$

and multiplying the letter codes times the powers yields

$$3,645 + 324 + 6$$

which sums to 3,975.

This process does indeed generate a unique number for every potential word.

You just calculated a 3-letter word. What happens with larger words?

Unfortunately, the range of numbers becomes rather large. The largest 10-letter word, zzzzzzzzzz, translates into

$$26*27^9 + 26*27^8 + 26*27^7 + 26*27^6 + 26*27^5 + 26*27^4 + 26*27^3 + 26*27^2 + 26*27^1 + 26*27^0$$

Just by itself, 27^9 is more than 7,000,000,000,000, so you can see that the sum will be huge. An array stored in memory can't possibly have this many elements, except perhaps, in some huge supercomputer. Even if it could fit, it

would be very wasteful to use all that memory to store a dictionary of just 50,000 words.

The problem is that this scheme assigns an array element to every potential word, whether it's an actual English word or not. Thus, there are cells reserved for *aaaaaaaaaa*, *aaaaaaaaab*, *aaaaaaaaac*, and so on, up to *zzzzzzzzzz*. Only a small fraction of these cells is necessary for real words, so most array cells are empty. This situation is illustrated in Figure 11-2. Near the word *elf*, there are several words that would be stored, such as *elk*, *eli* (for the given name *Eli*), and *elm*. The red arrows indicate a pointer to a record describing the word. At other places, such as around the word *bird*, there would be many unused cells, indicated by the cells without a pointer to some other structure.

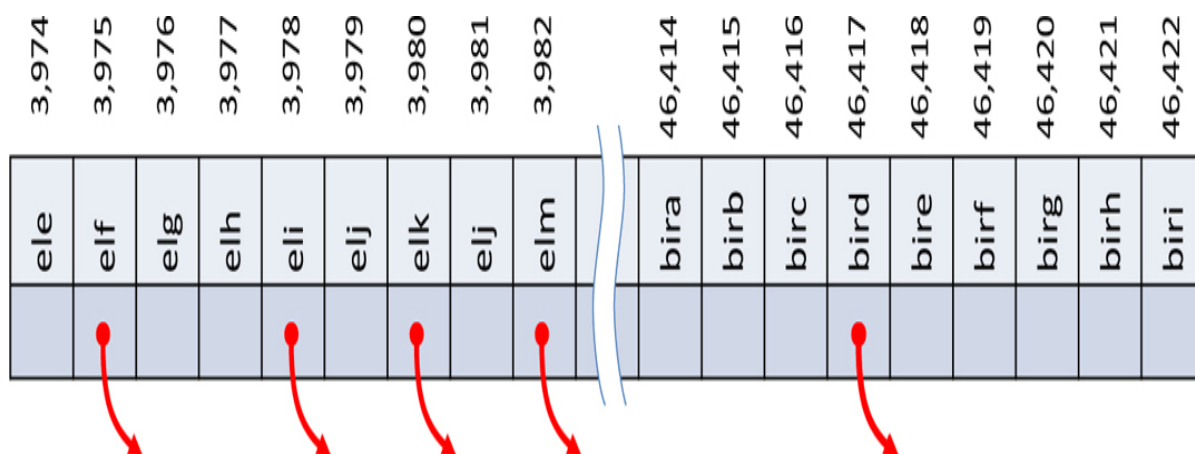


Figure 11-2 Index for every potential word

The first scheme—adding the numbers—generated too few indices. This latest scheme—adding the numbers times powers of 27—generates too many.

Hashing

What we need is a way to compress the huge range of numbers obtained from the numbers-multiplied-by-powers system into a range that matches a reasonably sized array.

How big an array are we talking about for this English dictionary? If you have only 50,000 words, you might assume the array should have approximately this many elements. It's preferable, however, to have extra cells, rather than too few, so you can shoot for an array twice that size, 100,000 cells. We discuss the advantages to having twice the minimum amount needed a little later.

Thus, we seek a way to squeeze a range of 0 to more than 7 trillion into the range 0 to 100,000. A simple approach is to use the modulo operator (%), which finds the remainder when one number is divided by another.

To see how this approach works, let's look at a smaller and more comprehensible range. Suppose you are trying to squeeze numbers in the range 0 to 199 into the range 0 to 9. The range of the big numbers is 200, whereas the smaller range has only 10. If you want to convert a big number (stored in a variable called `largeNumber`) into the smaller range (and store it in the variable `smallNumber`), you could use the following assignment, where `smallRange` has the value 10:

```
smallNumber = largeNumber % smallRange
```

The remainders when any number is divided by 10 are always in the range 0 to 9; for example, $13 \% 10$ gives 3, and $157 \% 10$ is 7. With decimal numbers, it simply means getting the last digit. The modulo operation compresses (or **folds**) a large range into a smaller one, as shown in [Figure 11-3](#). In our toy example, we're squeezing the range 0–199 into the range 0–9, which is a 20-to-1 compression ratio.

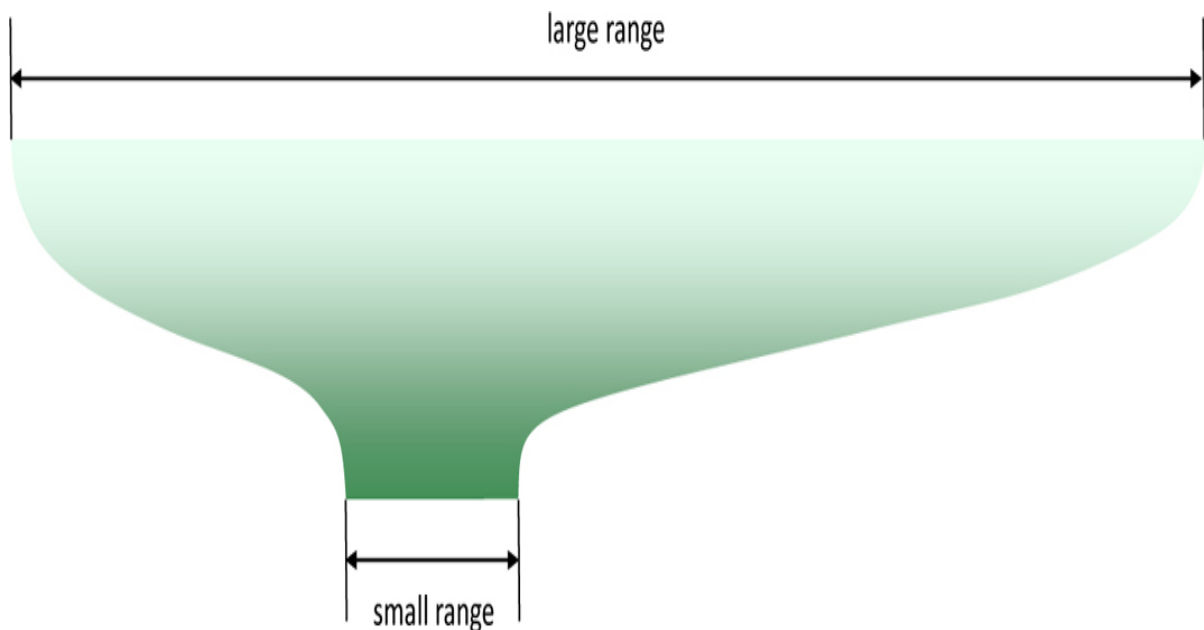


Figure 11-3 *Range conversion using modulo*

A similar expression can be used to compress the really huge numbers that uniquely represent every English word into index numbers that fit in the dictionary array:

```
arrayIndex = hugeNumber % arraySize
```

The function that computes the `hugeNumber` is an example of a **hash function**. It **hashes** (converts) a string (or some other piece of data) into a number in a large range. Taking the modulo with the `arraySize` maps the number into a smaller range. This smaller range is the range of index numbers in an array. The array into which data is inserted using a hash function is called a **hash Table**. The index to which a particular key maps is called the **hash address**. This terminology can be a little confusing. We use the term *hash table* to describe both the whole data structure and the array inside it that holds items.

To review: you can convert (or **encode**) a word into a huge number by converting each letter in the word into an integer and then multiplying them by an appropriate power of 27, based on their position in the word. [Listing 11-1](#) shows some Python code that computes the huge number.

Listing 11-1 *Functions to Uniquely Encode Simple English Words as Integers*

```
def encode_letter(letter): # Encode letters a thru z as 1 thru 26
    letter = letter.lower() # Treat uppercase as lowercase
    if 'a' <= letter and letter <= 'z':
        return ord(letter) - ord('a') + 1
    return 0 # Spaces and everything else are 0

def unique_encode_word_loop(word): # Encode a word uniquely using
    total = 0 # a loop to sum the letter codes times
    for i in range(len(word)): # a power of 27 based on their position
        total += encode_letter(word[i]) * 27 ** (len(word) - 1 - i)
    return total

def unique_encode_word(word): # Encode a word uniquely (abbreviated)
    return sum(encode_letter(word[i]) * 27 ** (len(word) - 1 - i)
               for i in range(len(word)))
```

The `encode_letter()` function takes a letter, gets its lowercase version, and checks whether it is in the range of 'a' to 'z', inclusive. If it is, it converts the letter to an integer by using Python's built-in `ord()` function. This function returns the Unicode value (also called *point*) of the character, which is the same as the ASCII value for the English letters. It returns the value of the character relative to the 'a' character ensuring that 'a' returns a value of 1. For

characters outside the range 'a' to 'z', it returns 0. That means that space is encoded as 0, as well as every other Unicode character that's not in the range.

To get the unique numeric code for a word, you can use a loop to sum up the values for each letter. The `unique_encode_word_loop()` function uses an index, `i`, into the letters of its `word` parameter to extract each one, get its encoded value using `encode_letter()`, multiply that value with a power of 27 appropriate for its position, and add the product to the running `total`. The power of 27 should be 0 for the last character of the `word`, which has the index `len(word) - 1`. For the second-to-last character at `len(word) - 2`, the exponent expression would be 1. The third-to-last would be exponent 2, and so on, up to exponent `len(word) - 1` for the first character (leftmost) in the word. After the loop exits, the `total` is returned.

[Listing 11-1](#) also shows a `unique_encode_word()` function that computes the exact same encoded value. It calculates it, however, using a more compact syntax with a list comprehension. The `sum()` function returns the sum of its arguments. The list (tuple) comprehension provides the arguments to `sum()`. Comprehensions are in the form

expression for variable in sequence

and in the `unique_encode_word()` function, `i` is used as the index *variable* that comes from the comprehension *sequence* (which are the indices of letters in `word`). The *expression* is the same as what was used in the loop version.

The `unique_encode_word()` function is an example of a hash function. Using the modulo operator (%), you can squeeze the resulting huge range of numbers into a range about twice as big as the number of items you want to store. This computes a hash address:

```
arraySize = numberWords * 2
arrayIndex = unique_encode_word(word) % arraySize
```

In the huge range, each number represents a potential data item (an arrangement of letters), but few of these numbers represent actual data items (English words). A hash address is a mapping from these large numbers into the index numbers of a much smaller array. In this array you can expect that, on the average, there will be one word for every two cells. Some cells will have no words, some will have one, and there can be others that have more than one. How should that be handled?

Collisions

We pay a price for squeezing a large range into a small one. There's no longer a guarantee that two words won't hash to the same array index.

This is similar to the problem you saw when the hashing scheme was the sum of the letter codes, but the situation is nowhere near as bad. When you added the letter codes, there were only 260 possible numeric values (for words up to 10 letters). Now you're spreading the codes over the 100,000 possible array cells.

It's impossible to avoid hashing several different words into the same array location, at least occasionally. The plan was to have one data item per index number, but this turns out not to be possible in most hash tables. The best you can do is hope that not too many words will hash to the same index.

Perhaps you want to insert the word *abductor* into the array. You hash the word to obtain its index number but find that the cell at that number is already occupied by the word *bring*, which happens to hash to the exact same number. This situation, shown in [Figure 11-4](#), is called a **collision**. The word *bring* has a unique code of 1,424,122, which is converted to 24,122 by taking the modulo with 100,000. The word *abductor* has the unique code 11,303,824,122, and *missable* has 139,754,124,122. All three of them hash to index 24,122 of the hash table.

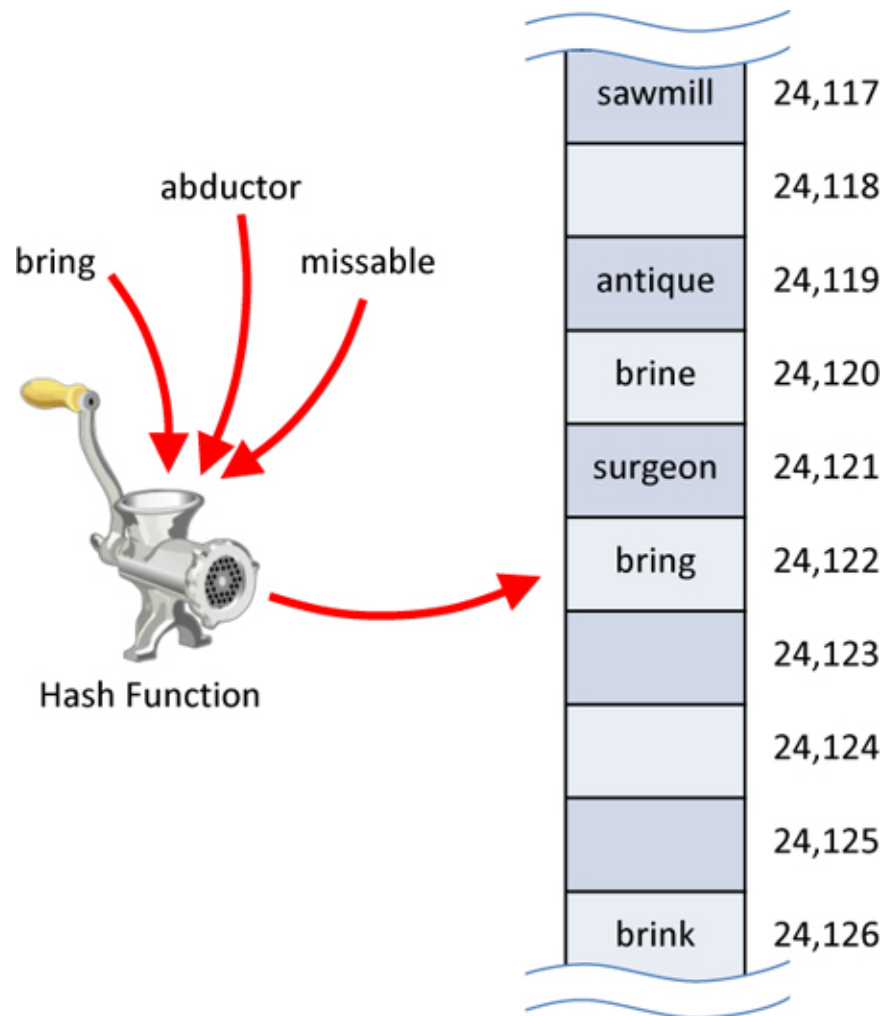


Figure 11-4 A collision

The slightly different words *brine* and *brink* hash to locations nearby the cell for *bring*. The reason is that they differ only in their last letter, and that letter's code is multiplied by 27^0 , or 1. Other words could also hash to those same locations.

How Bad Are Collisions?

Is the hash function a good idea? Could running strings of letters through a “grinder” ever produce some kind of useful hash? It may appear that the possibility of collisions renders the scheme impractical. It would help to know how often they are likely to occur in designing strategies to deal with them.

One relevant measure can be seen by answering a classic question: when is it more likely than not that two people at a gathering share the same birth day and

month? [Figure 11-5](#) illustrates the concept. At first, that idea might not seem relevant to hash tables. On closer inspection, you can think of the days in a year as the cells of a hash table. Each birthday lands in exactly one of them.

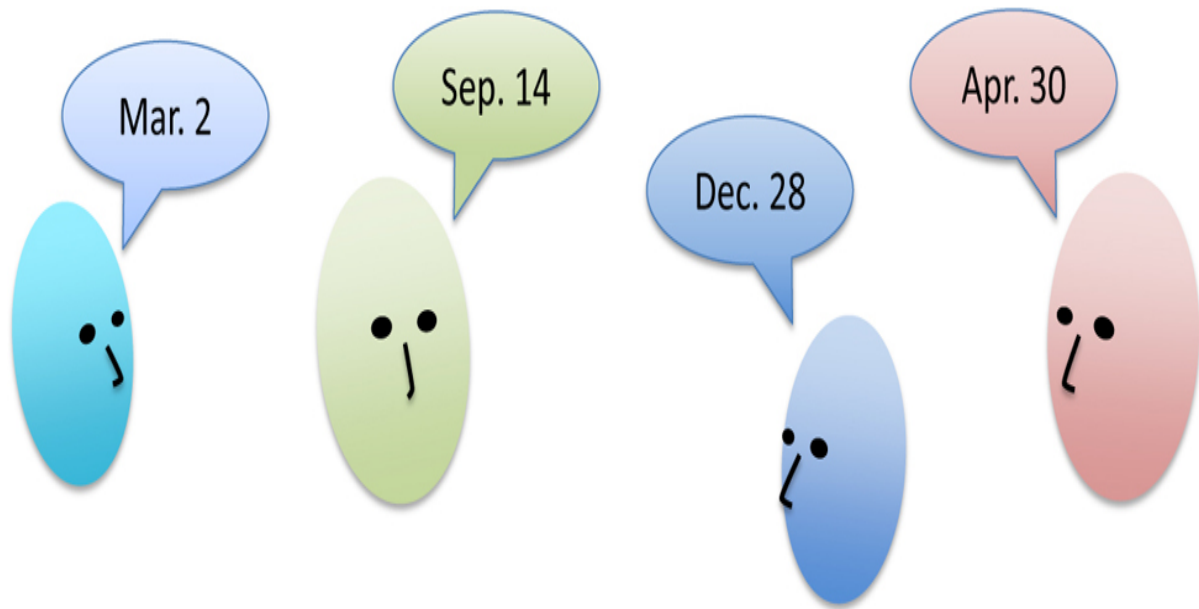


Figure 11-5 *Finding shared birthdays at a gathering*

If there are only a few people at the gathering, it's highly unlikely that they share a birth day. If there are 367 people, then it is certain that some have the same birth day. Somewhere between those extremes, there's a number of people where the likelihood of a shared birthday is greater than the likelihood that they are all different.

Maybe intuition might tell you that if you have half as many people as there are days in the year, then the likelihood would be greater than 50 percent for a shared birth day. In other words, if there are 183 people, it is more likely than not to have a shared birth day. That intuition is correct, but the point at which it changes from below 50 percent to above 50 percent is at 23 people. With 22 people, it's still more likely they all were born on different days. This calculation assumes that the birthdays are distributed randomly throughout the year (which is not the case). At a gathering for people born under a particular sign of the zodiac, of course, the distribution would be very different!

So even when the ratio of items to hash table cells is less than 10 percent (23 out of 366), the chance of a collision is greater than 50 percent. That means you should plan your hash tables to always deal with collisions. You can work around the problem in a variety of ways.

We already mentioned the first technique: specifying an array with at least twice as many cells as data items. This means you expect half the cells to be empty. One approach, when a collision occurs, is to search the array in some systematic way for an empty cell and insert the new item there, instead of at the index specified by the hash address. This approach is called **open addressing**. It's somewhat like boarding a train or subway car; you enter at one point and take the nearest seat that's open. If the seats are full, you continue through the car until you find an empty seat. The seat closest to the door where you entered is analogous to the initial hash address.

Returning to hashing words into numbers, if *abductor* hashes to 24,122, but this location is already occupied by *bring*, then you might try to insert *abductor* in 24,123, for example. When the insert operation finds an empty cell, it stores *both the key and its associated value*. In that way, search operations using open addressing can compare the original keys to the keys stored in the table to determine how far the search should continue. It also enables easy checking for empty cells because they won't have a key-value structure.

A second approach (mentioned earlier) is to create an array that consists of references to another data structure (like linked lists of words) instead of the records for the individual words. Then, when a collision occurs, the new item is simply inserted in the list at that index. This is called **separate chaining**.

In the balance of this chapter, we discuss open addressing and separate chaining, and then return to the question of hash functions.

So far, we've focused on hashing strings. In practice, many hash tables are used for storing strings. Hashing by birthdays is certainly possible, but only useful in rare instances. Many other hash tables are keyed by numbers, as in the bank account number example, or in the case of credit card numbers. In the discussion that follows, we use numbers—rather than strings—as keys. This approach makes things easier to understand and simplifies the programming examples. Keep in mind, however, that in many situations these numbers would be derived from strings or byte sequences.

Open Addressing

In open addressing, when a data item can't be placed at the index calculated by the hash address, another location in the array is sought. We explore three methods of open addressing, which vary in the method used to find the next

vacant cell. These methods are *linear probing*, *quadratic probing*, and *double hashing*.

Linear Probing

In **linear probing** the algorithm searches sequentially for vacant cells. If cell 5,421 is occupied when it tries to insert a data item there, it goes to 5,422, then 5,423, and so on, incrementing the index until it finds an empty cell. This operation is called linear probing because it steps sequentially along the line of cells.

The HashTableOpenAddressing Visualization Tool

The HashTableOpenAddressing Visualization tool demonstrates linear probing. When you start this tool, you see a screen like that in [Figure 11-6](#).

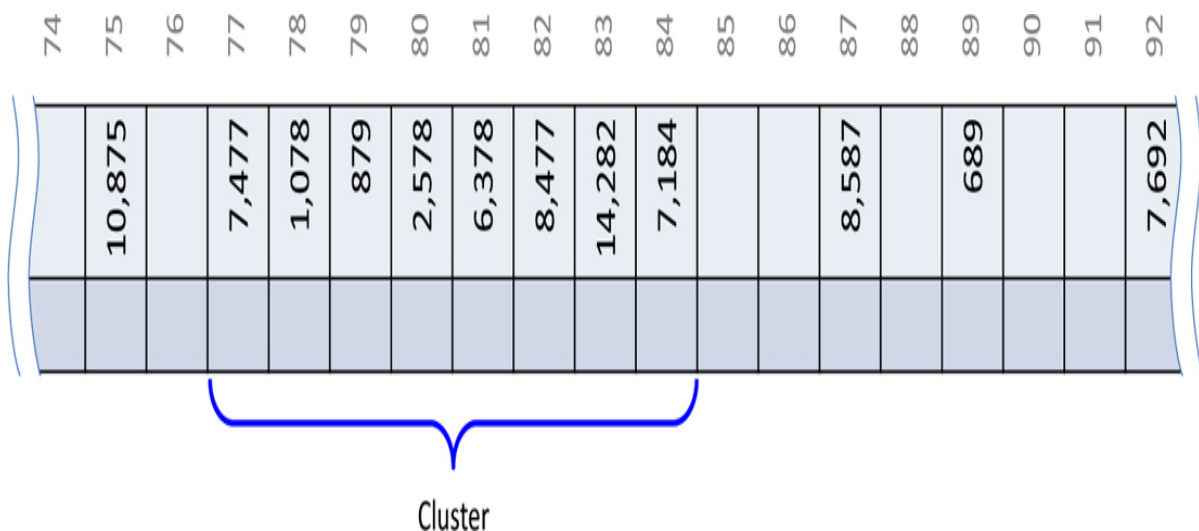


Figure 11-6 The HashTableOpenAddressing Visualization tool at startup

In this tool, keys can be numbers or strings up to 8 digits or characters. The initial size of the array is 2. The hash function has to squeeze the range of keys down to match the array size. It does this with the modulo operator (%), as you've seen before:

```
arrayIndex = key % arraySize
```

For the initial array size of 2, this is

```
arrayIndex = key % 2
```

This hash function is simple, so you can predict what cell will be indexed. If you provide a numeric key to one of the operations, the key hashes to itself, and the modulo 2 operation produces either array index 0 or 1. For string keys (anything that contains characters other than the decimal digits), it behaves similar to the `unique_encode_word()` function shown in [Listing 11-1](#). For example, the key *cat* hashes to 7627107, which produces index 1. The key *bat* hashes to one less, 7627106, which produces index 0.

A two-cell hash table can't hold much data, obviously, and soon you'll see what happens as it begins to fill up. The number of cells is shown directly below the last cell of the table, and the cell indices appear to the left of each cell. The current number of items stored in the hash table, `nItems`, is shown in the center.

The box labeled “HASH” represents the hashing function. Let's see how new keys are processed by it and used to find array indices.

The Insert Button

To put some data in the hash table, type a key, *cat* for example, in the top text entry box and select Insert. The visualization tool shows the process of passing the string “cat” through the hashing function to get a big integer. Using the modulo of the table size, it determines a cell index and draws an arrow connecting the hashed result to it like the one shown in [Figure 11-7](#). The arrow points to where probing will begin to find an empty cell. Because the table is initially empty, the first probe finds an empty cell, and the insert operation finishes by copying the key into it—along with a colored background representing some associated data—and then incrementing the `nItems` value.



Figure 11-7 *Probing to insert the key cat in an empty hash table*

The next item inserted can show what happens in a collision. Inserting the key *eat* causes a hash address of 7627109, which probes cell 1, as shown in [Figure 11-8](#).



Figure 11-8 *Probing to insert the key eat*

After finding cell 1 occupied, the insertion process begins probing each cell sequentially—linear probing—to find an empty cell, as shown with the additional curved arrow in [Figure 11-8](#). The probing would normally start at index 2, but because that index lies beyond the end of the table, it wraps around to index 0. Because cell 0 is empty, the key *eat* can be stored there along with its associated data.

After incrementing the `nItems` value to 2, the table is now full. To be able to add more items in the future, the visualization tool shows what happens next. A new table is allocated that is at least twice as big. The items from the old table are then reinserted in the new table by **rehashing** them. The hashing function hasn't changed, nor have the keys, so it might appear that the items would end up in their same relative positions. Because the size of the table grew, however, the modulo operator produces new cell indices. The *cat* and *eat* keys end up in cells 3 and 5 this time, as shown in [Figure 11-9](#).



Figure 11-9 *After inserting cat and eat in an empty hash table*

We explore the details of this process in the “[Growing Hash Tables](#)” and “[Rehashing](#)” sections later. First, let’s explore more about the visualization tool and linear probing.

The Random Fill Button

Initially, the hash table starts empty and grows as needed. To explore what happens when larger tables get congested, you fill can them with a specified number of data items using the Random Fill button. Try entering 2 in the text entry box and selecting Random Fill. The visualization tool generates two random strings of characters as keys and animates the process of inserting them.

The animation process takes some time, and when you understand how the insertions work, it may be preferable to jump right to the end result. If you uncheck the button labeled Animate Hashing, the Random Fill operation will perform all the insertions without animation. Similarly, single item inserts will skip the animation of hashing the key (but not of the probing that happens afterward). Try disabling the animation and inserting 11 more items. You’ll see that as the table grows, it divides into multiple columns, as shown in the example of [Figure 11-10](#).



Figure 11-10 *A hash table with 15 items*

The Search Button

To locate items within the hash table, you enter the key of the item and select the Search button. If the Animate Hashing button is checked, the tool animates the conversion of the key string to a large number. The probing of the table begins with the index determined from the hashed key. If it finds the cell filled and the key matches, the key and the color representing its data are copied to an output box.

The visualization tool simplifies searching for randomly generated and other existing keys by copying the key to the text entry box when a stored key is clicked. The search behavior gets a little more complex when the key isn't in the table. The tool uses a hashing function that treats numeric keys specially: they hash to their numeric value. Try typing 3 for the key (or clicking the index of another empty cell of a table like the one in [Figure 11-10](#)) and selecting Search. The initial probe lands on an empty cell, and the tool immediately discovers that the item is not in the table.

Now try entering the index of a filled cell, like 14 in [Figure 11-10](#). You can also click the index number, but be sure that the key is the numeric index and not the string key stored in the cell. When you select Search, the visualization tool shows the initial probe going to the selected index. Finding the cell full, but not containing the desired key, it starts linear probing to see whether a collision happened when the item was inserted. The next empty cell probed ends the search.

Filled Sequences and Clusters

As you might expect, some hash tables have items evenly distributed throughout the cells, and others don't. Sometimes there's a sequence of several empty cells and sometimes a sequence of filled cells. In the example of [Figure 11-10](#), the filled sequences comprise four 1-item sequences, one 4-item sequence, and one 7-item sequence.

Let's call a sequence of filled cells in a hash table a **filled sequence**. As you add more and more items, the filled sequences become longer. This phenomenon is called **clustering** and is illustrated in [Figure 11-11](#). Note that the order that items were inserted into the table determines how far away a key is placed relative to its default location.



Figure 11-11 *An example of clustering in linear addressing*

When you're searching for a key, it's possible that the first indexed cell is already occupied by a data item with some other key. This is a collision; you see the visualization tool add another arrow pointing to the next cell. The process of finding an appropriate cell while handling collisions is called **probing**.

Following a collision, the hash table's search algorithm simply steps along the array looking at each cell in sequence. If it encounters an empty cell before finding the goal key, it knows the search has failed. There's no use looking further because the insertion algorithm would have inserted the item at this cell (if not earlier). [Figure 11-12](#) shows successful and unsuccessful linear probes in a simplified hash table. By *simplified*, we mean that it uses the last two digits of the key as the table index, which is not a good idea in practice. (You see why a little later.) The initial probe for key 6,378 lands at cell 78. It probes the next adjacent cells until it finds the matching key in cell 81. The search for key 478

also starts at cell 78. After probing 7 cells in the filled sequence, it finds an empty cell at index 85, which ends the search.

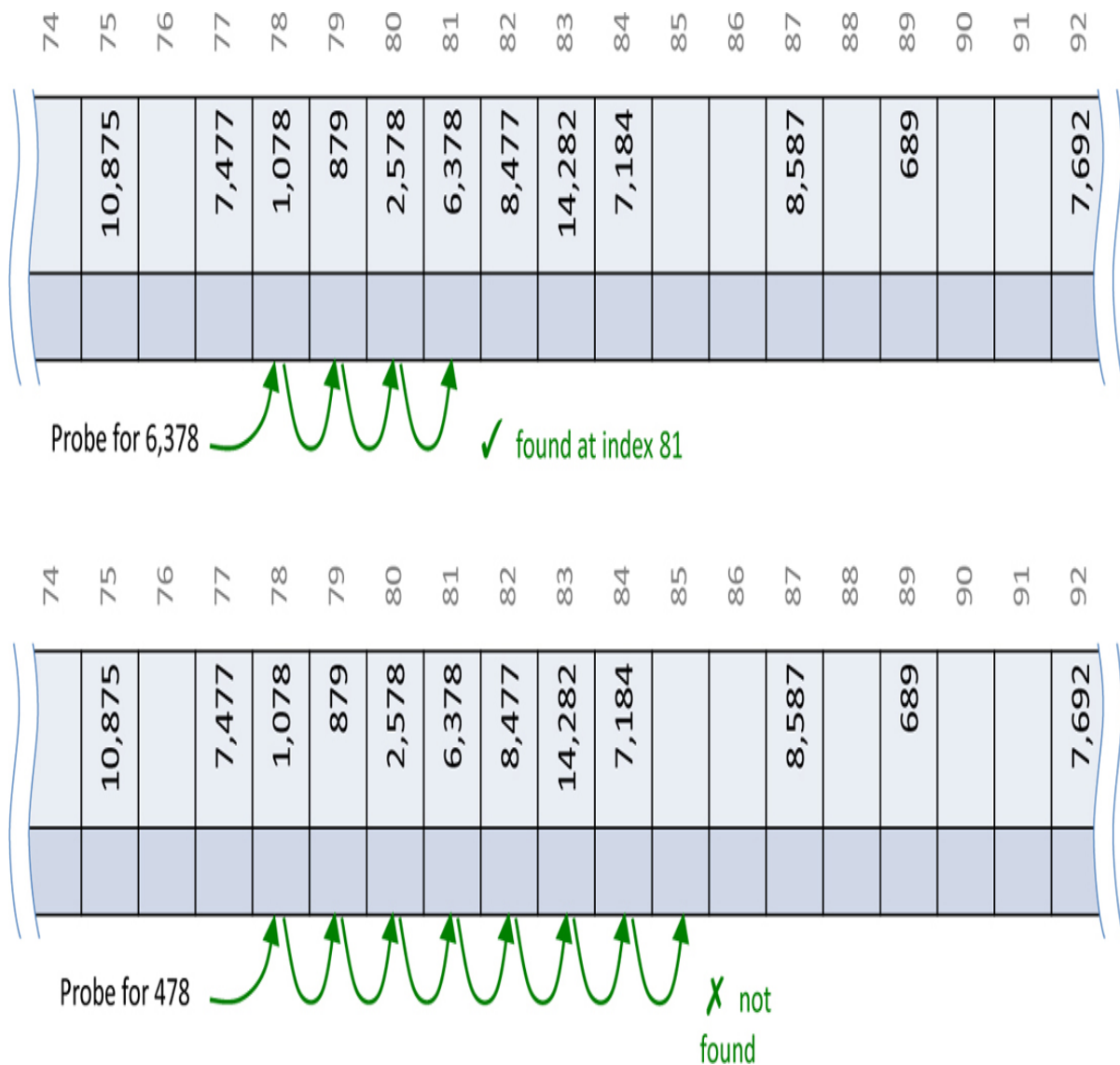


Figure 11-12 *Linear probes in clusters*

Try experimenting with filled sequences. Find the starting index of such a sequence like index 26 in [Figure 11-10](#). After clicking that index to copy it into the text entry box, select Insert (not Search). The insertion algorithm must step through all of the filled cells to find the next empty one. After it's inserted, if you now search for that same index, the search must repeat that same process.

The Delete Button

The Delete button deletes an item whose key is typed by the user. Deletion isn't accomplished by simply removing a data item from a cell, leaving it empty. Why not? We look at the reason a little later. For now, you can see that the tool replaces the deleted item with a special key that appears as DELETED in the display.

The Insert button inserts a new item at the first available empty cell or in a deleted item. The Search button treats a deleted item as an existing item for the purposes of searching for another item further along.

If there are many deletions, the hash table fills up with these ersatz data items, which makes it less efficient. For this reason, some open addressing hash table implementations don't allow deletion. If it is implemented, it should be used sparingly to avoid large amounts of unused memory.

Duplicates Allowed?

Can you allow data items with duplicate keys to be used in hash tables? The visualization doesn't allow duplicates, which is typical behavior for hash tables. As mentioned in previous chapters, this approach implements the storage type as an associative array, where each key can have at most one value.

The alternative of allowing duplicate keys would complicate things. It would require rewriting the search algorithm to look for *all* items with the same key instead of just the first one, at least in some circumstances. That requires searching through all filled sequences of cells until an empty cell is found. Probing the entire filled sequence wastes time for all table accesses, even when no duplicates are present. Deleting an item would either try to delete the first instance or all instances of a particular key. Both cases require probing filled sequences to find the extent of the duplicates and then moving at least one of the items in the sequence to their default positions if the deletions opened up some cells. For these reasons, you probably want to forbid duplicates or use another data structure if they are required.

Avoiding Clusters

Try inserting more items in the HashTableOpenAddressing Visualization tool. The tool stops growing the table when it reaches 61 cells. As the table gets fuller, the clusters grow larger. Clustering can result in very long probe lengths. This means that accessing cells deeper in the sequence is very slow.

The fuller the array is, the worse clustering becomes. For an extreme example, use the Random Fill button to enter enough random keys so that the total number of keys is 60. Now try searching for a key that's not in the table. The initial probe lands somewhere in the 61-cell array and then hunts for the single remaining empty (or possibly deleted) cell. If you are unfortunate enough for the initial probe to be the cell after the empty one, the search can go through all 61 cells.

Clustering is not usually a problem when the array is half full and still not too bad when it's two-thirds full. Beyond this, however, performance degrades seriously as the clusters grow larger and larger. For this reason, it's critical when designing a hash table to ensure that it never becomes more than half, or at the most two-thirds, full. We discuss the mathematical relationship between how full the hash table is and probe lengths at the end of this chapter.

Clusters are created during insertion but also affect deletion. When an item is deleted from a hash table, you would ideally mark its cell as empty so that it can be used again for a later insert and to break up potential clusters. That simple strategy, however, is a problem for open addressing because probes follow a sequence of indices in the table to locate items, stopping when they find empty cells. If you delete an item that happened to be in the middle of such a sequence, such as item 879 or item 2,578 in [Figure 11-12](#), for example, items landing later in the probe sequence, such as item 6,378, would not be found by subsequent searches. Missing items are particularly problematic when the item being deleted is part of multiple probe sequences. In the example, item 879 happens to be in the probe sequence for item 6,378 even though their sequences start at different table indices.

You might think that there is some way to rearrange items to fill the hole created by a deletion. For instance, what if you follow the probe sequence to its end. Couldn't you move the last item in the sequence to fill the cell being deleted (somewhat like the successor node replacing a deleted node in a binary search tree)? Unfortunately, that last item in the sequence could be an item with a key that doesn't hash to the start of the probe sequence used to find the item to be deleted. For example, if you were deleting item 1,078 in [Figure 11-12](#), following its probe sequence to the end would suggest that item 7,184 could replace it, but that would cause that item to be lost from its normal probe sequence that starts at cell 84.

Another idea: what if you find the last item in the deletion probe sequence that shares the same starting cell as the item being deleted? That ensures that you

only shift an item on the same probe sequence. Unfortunately, that too causes problems because it could create a new hole in another probe sequence. If some other probe sequence happens to land on the item that was moved, then that sequence is broken. You could try to find any sequences that would visit the cell holding the item being moved as they skipped past collisions, but there could be many, many such sequences. Just like with automobiles, cleaning up after collisions is a big problem.

The approach for deleting items in open addressing is to simply mark cells as deleted, as the Visualization tool shows. The search algorithm can then step past deleted cells as it probes for a key. The insert algorithm, too, can hunt for either empty or deleted cells to use for new items. That helps a bit in keeping the size of clusters small, but only for insertion. You still must search past deleted cells when seeking an item (for search or delete). It also wastes memory, as we discuss later.

Python Code for Open Addressing Hash Tables

Let's look at the implementation of open addressing in hash tables. Aside from some of the fancier hashing functions, they are straightforward to implement. We'll make a class where it's easy to change the hash function and probing technique to resolve collisions. That design choice makes exploring different options more convenient, but it's not particularly good for performance.

The Core HashTable

The hash table object must maintain an array to hold the items it stores. That table should be private because callers should not be able to manipulate its entries. How big should that table be? We can choose to provide a parameter in the constructor to set the size, but like other data structures, it will be useful to allow it to expand later if more cells are needed.

Because we're making one class to handle hash tables with different open addressing probes, the constructor also needs a way to specify the method to search when collisions are found. The code in [Listing 11-2](#) handles that feature by providing a `probe` parameter. The default value for the `probe` is `linearProbe`, which we describe shortly. There are also parameters for the initial `size` of the table, the `hash` function, and a `maxLoadFactor`, explained later.

Listing 11-2 *The Core HashTable Class*

```
class HashTable(object):    # A hash table using open addressing
    def __init__(           # The constructor takes the initial
        self, size=7,      # size of the table,
        hash=simpleHash,   # a hashing function,
        probe=linearProbe, # the open address probe sequence, and
        maxLoadFactor=0.5): # the max load factor before growing
        self.__table = [None] * size # Allocate empty hash table
        self.__nItems = 0            # Track the count of items in the table
        self.__hash = hash           # Store given hash function, probe
        self.__probe = probe          # sequence generator, and max load factor
        self.__maxLoadFactor = maxLoadFactor

    def __len__(self):        # The length of the hash table is the
        return self.__nItems # number of cells that have items

    def cells(self):          # Get the size of the hash table in
        return len(self.__table) # terms of the number of cells

    def hash(self, key):      # Use the hashing function to get the
        return self.__hash(key) % self.cells() # default cell index
```

The constructor creates a private `__table` of the specified size. The initial `None` value in the cells indicates that they are empty. The count of stored items, `__nItems`, is set to zero. When items are inserted, they place (key, value) tuples in the table's cells, making empty and full cells easy to distinguish. All of the rest of the constructor parameters are stored in private fields for later use.

The `HashTable` defines a `__len__()` method so that Python's `len()` function can be used on instances to find the number of items they contain. A separate `cells()` method returns the number of cells in the table, so you can see how full the table is by comparing it to the number of items. As it fills, the likelihood of collisions increases.

The other core method shown in [Listing 11-2](#) is the `hash()` method. This method is used to hash keys into table indices. We've allowed the caller to provide the hashing function in the constructor. This could be the `unique_encode_word()` function from [Listing 11-1](#) or something similar. Whatever function is used, it should return an integer from a single key argument. The modulo of that integer with the number of cells in the table

provides the initial table index for that key. This design allows callers to provide hashing functions that return very large integers, which are then mapped to the range of cells in the table.

The simpleHash() Function

The default hash function for the `HashTable` is `simpleHash()`, which is shown in [Listing 11-3](#). This function accepts several of the common Python data types and produces an integer from their contents. It's not a sophisticated hashing function, but it serves to show how such functions are created to handle arbitrary data types.

Listing 11-3 *The simpleHash() Method*

```
def simpleHash(key):          # A simple hashing function
    if isinstance(key, int):  # Integers hash to themselves
        return key
    elif isinstance(key, str): # Strings are hashed by letters
        return sum(
            256 ** i * ord(key[i]) # 256 to the power of its position
            for i in range(len(key)) # in the string
        )
    elif isinstance(key, (list, tuple)): # For sequences,
        return sum(
            256 ** i * simpleHash(key[i]) # by 256 to the power of its
            for i in range(len(key)) # position in the sequence
        )
    raise Exception(          # Otherwise it's an unknown type
        'Unable to hash key of type ' + str(type(key)))
```

The `simpleHash()` function checks the type of its `key` argument using Python's `isinstance()` function. For integers, it simply returns the integer. Returning the unmodified key is, in general, a *very bad idea* because many applications use a hash table on integers in a small range. That small range (or distribution) of numbers will map directly to a small range of cell indices in the table and likely cause collisions. We choose to use it here to simplify the processing, experiment with collisions, and look at ways to avoid them.

If the key passed to `simpleHash()` is a string, the resulting integer it produces is something like the `unique_encode_word()` function you saw earlier. It takes the numeric value of each character in the key using `ord()` and multiplies that by a power of 256. The power is the position of the character in the string. The first character is power 0, which multiplies its numeric `ord` value by 1. The

second character gets power 1, so it is multiplied by 256^1 , the third character is multiplied by 256^2 , and so on. The multiplication scheme ensures that anagram strings like "ant" and "tan" will map to different values, at least for simple strings. The products are all summed together using `sum()` and a tuple comprehension.

Note that the use of powers of 256 in the `simpleHash()` method is not sufficient to distinguish all string values. Because Python strings may contain any Unicode character whose numeric value can range up to $0x10FFFF = 1,114,111$, the `simpleHash()` function can hash different strings to the same integer. Using 1,114,112 as the base instead of 256 avoids that problem, but we use 256 to keep the numbers smaller in our examples at the risk of causing more collisions.

The last `elif` clause in `simpleHash()` handles lists and tuples. These are the simple sequence types in Python. They are like strings, except that their elements could be any other kind of data, not just Unicode characters. It applies the same multiplication scheme by recursively calling `simpleHash()` on the elements individually. In this way, `simpleHash()` can recursively descend through a complex structure of sequences to find the integers and strings they contain, and build a number based on them and their relative positions.

Finally, if none of the data type tests succeed, `simpleHash()` gives up and raises an exception to signal that it doesn't have a method to convert it to an integer.

The search() Method

The `search()` method is used to find items in the hash table, navigating past any collisions. It does this by calling an internal `__find()` method to get the table index for the key as shown in [Listing 11-4](#). It's best to keep that method private because callers shouldn't need to know which cell holds a particular item.

Listing 11-4 *The `search()` and `__find()` Methods of `HashTable`*

```
class HashTable(object):    # A hash table using open addressing
...
    def search(self,        # Get the value associated with a key
               key):        # in the hash table, if any
```



```

i = self.__find(key) # Look for cell index matching key
return (None if (i is None) or # If index not found,
        self.__table[i] is None or # item at i is empty or
        self.__table[i][0] != key # it has another key, return
        else self.__table[i][1]) # None, else return item value

__Deleted = (None, 'Deletion marker') # Unique value for deletions

def __find(self,          # Find the hash table index for a key
            key,          # using open addressing probes. Find
            deletedOK=False): # deleted cells if asked
    for i in self.__probe(self.hash(key), key, self.cells()):
        if (self.__table[i] is None or # If we find an empty cell or
            (self.__table[i] is HashTable.__Deleted and # a deleted
             deletedOK) or # cell when one is sought or the
            self.__table[i][0] == key): # 1st of tuple matches key,
            return i                  # then return index
    return None                      # If probe ends, the key was not found

```

The `__find()` method returns an integer index to a cell or possibly `None` when it cannot find the key being sought. The `search()` method looks at the returned index and returns `None` in the cases where the key wasn't found. In other words, if the index returned by `__find()` is `None` or the table cell it indicates contains `None`, or the key stored in that table cell is not equal to the key being sought, the search for the key failed. The only other possibility is that the table cell's key matches the one being sought, so it returns the second item in the cell's tuple, the value associated with the key.

The definition of the constant, `__Deleted`, might seem a little unusual. This is the value stored in table cells that have been filled and later deleted. It's a **marker value**. By making it a tuple in Python, it has a unique reference address that can be compared using the `is` operator. The code must distinguish between empty cells containing `None`, deleted cells containing `__Deleted`, and full cells during open address searching. The comparison test in the `__find()` method (described shortly) uses the `is` operator instead of the `==` operator to compare cell contents with `__Deleted` in case some application decided to store a copy of that same tuple. The `search()` method doesn't care whether the cell returned by `__find()` is empty or deleted, but the `insert()` method does, as you see shortly. Note also that the `__Deleted` marker's key, `None`, cannot be hashed by `simpleHash()`. If it could, then the `search()` method might return the deleted marker value as a result.

The `__find()` method takes the search key as a parameter with an optional flag parameter, `deletedOK`, that tells it whether it can stop after finding a deleted cell. This method implements the core of the open addressing scheme. The key is hashed using the hash function that was provided when the table was created. The `hash()` method ([Listing 11-2](#)) is called to map the large integer computed by `simpleHash()` or some other hash function to an integer in the range of the current size of the hash table. That hash address is the starting point for probing the cells of the table for the item.

The hash address returned by the call to `hash()` is passed to the probe function that was given when the hash table was constructed. The loop

```
for i in self.__probe(self.hash(key), key, self.cells()):
```

shows that the probe function is being used as a generator. In other words, it must create an iterator that iterates over a sequence. The elements of the sequence are the table cell indices that should be probed for the item. The call to `self.hash(key)` returns the first index, and the key and number of cells arguments allow the generator to know how to create the rest of the sequence. We look at the `linearProbe()` generator definition shortly, but first let's look at the rest of the `__find()` method.

Inside the `for` loop, `__find()` checks the contents of cell `i` to see what's stored there. If it's `None`, the cell is empty and `i` can be returned to indicate the key is not in the table. If the cell isn't empty and has a matching key, then `__find()` can also return `i` as the result to indicate the item was found. The only tricky case is what to do if the cell has been marked as deleted. The default (`deletedOK=False`) is to treat it like another filled cell caused by a collision and continue the probe sequence. Only if the caller asked to stop on deleted cells, and the cell's value is the `__Deleted` marker, will `__find()` end the loop and return.

When some other item is found at cell `i`, the probe sequence continues. For linear probing, that is just index `i+1` or 0, after it reaches the number of cells in the table. If the whole probe sequence is completed without finding any empty cells, then the table must be full of nonmatching or deleted items. In that case, `__find()` returns `None`.

The insert() Method

The process of inserting items in the table follows the same scheme as searching and adds a few twists for handling the increasing number of items.

Listing 11-5 shows the `insert()` method getting the index of a cell, `i`, by calling `__find()` on the key of the item to insert. The call is made with `deletedOK=True` to allow finding deleted cells, which `insert()` will fill.

Listing 11-5 *The `insert()` and `__growTable()` Methods*

```
class HashTable(object):    # A hash table using open addressing
...
    def insert(self,        # Insert or update the value associated
               key, value): # with a given key
        i = self.__find(    # Look for cell index matching key or an
            key, deletedOK=True) # empty or deleted cell
        if i is None:       # If the probe sequence fails,
            raise Exception( # then the hash table is full
                'Hash table probe sequence failed on insert')
        if (self.__table[i] is None or # If we found an empty cell, or
            self.__table[i] is HashTable.__Deleted): # a deleted cell
            self.__table[i] = ( # then insert the new item there
                key, value)     # as a key-value pair
            self.__nItems += 1 # and increment the item count
            if self.loadFactor() > self.__maxLoadFactor: # When load
                self.__growTable() # factor exceeds limit, grow table
            return True         # Return flag to indicate item inserted

        if self.__table[i][0] == key: # If first of tuple matches key,
            self.__table[i] = (key, value) # then update item
            return False             # Return flag to indicate update

    def loadFactor(self):    # Get the load factor for the hash table
        return self.__nItems / len(self.__table)

    def __growTable(self):   # Grow the table to accommodate more items
        oldTable = self.__table # Save old table
        size = len(oldTable) * 2 + 1 # Make new table at least 2 times
        while not is_prime(size): # bigger and a prime number of cells
            size += 2             # Only consider odd sizes
        self.__table = [None] * size # Allocate new table
        self.__nItems = 0          # Note that it is empty
        for i in range(len(oldTable)): # Loop through old cells and
            if (oldTable[i] and # insert non-deleted items by re-hashing
                oldTable[i] is not HashTable.__Deleted):
                self.insert(*oldTable[i]) # Call with (key, value) tuple
```

The first test on `i` checks whether it is `None`, indicating that the probe sequence ended without finding the key, an empty cell, or a deleted cell. Either the table is full, or the probe sequence has failed to find any available cells in this case. The `insert()` method raises an exception for that. The method could try increasing the table size for this situation, but if there's a problem with the probe sequence, increasing the table size may only make matters worse.

The next test checks whether cell `i` is empty or deleted. In those cases, the contents can be replaced by a (key, value) tuple to store the item in the cell. Doing so adds a new item to the table, and the `insert()` method increments the number of items field. That increase could make the table full or nearly full. To reduce the problem of collisions, the method should increase the size of the table when the number of items exceeds some threshold.

What threshold should be used? An absolute number doesn't make sense because when it's surpassed, the table could become full again. Instead, it's better to look at the **load factor**, the ratio (or percentage) of the table cells that are full. The `load_factor()` method computes the value, which is always a number in the range 0.0 to 1.0. By comparing the load factor to the `maxLoadFactor` specified when the hash table was constructed, we can use a single threshold that's valid no matter how large the hash table grows. We examine the `__growTable()` method shortly.

The `insert()` method finishes by returning `True` when an empty or deleted cell becomes filled. This value indicates to the caller that another cell became full. The alternative, when the hash table already has a value associated with the key to be inserted, is to replace or update the value with the new one. The final `if` clause of the `insert()` method returns `False` to indicate that no unused cells were filled by the insertion.

Growing Hash Tables

The `__growTable()` method in [Listing 11-5](#) increases the size of the array holding the cells. We explored growing arrays in one of the Programming Projects from [Chapter 2](#), “[Arrays](#),” and the process is a bit more complicated for hash tables. First, let's look at how much it should grow. We could add a fixed amount of cells or multiply the number of cells by some growth factor. Adding a small, fixed number of cells would keep the number of unused cells to a minimum. Multiplying by, say 2, creates a large number of unused cells

initially, but means that the grow operation will be performed many fewer times.

To see the difference the growth method has, let's assume that the application using the hash table chooses to start with a small hash table of five cells and that it must store 100,000 key-value pairs. If the choices are to grow the table by five more cells or double its size for each growth step, how many steps will be needed? [Table 11-1](#) shows the steps in growing the size of the table for the two methods.

Table 11-1 *Growing Tables by a Fixed Increment and by Doubling*

Fixed Size Growth		Doubling Size Growth	
Step	Size	Step	Size
0	5	0	5
1	10	1	10
2	15	2	20
3	20	3	40
4	25	4	80
...		...	
N	$5 * (N + 1)$	N	$5 * 2^N$
...		...	
14	75	14	81,920
15	80	15	163,840
...			
19,998	99,995		
19,999	100,000		

The fixed size growth takes 20,000 steps to reach the 100,000 cells needed. When the size doubles at every step, the 100,000 capacity is reached on the 16th step. As you've seen before, that is the difference between $O(N)$ and $O(\log N)$ steps. Reducing the number of growth steps is important because of what must be done after growing the array. Before we look at that, however, there's another factor in choosing the size of the new array.

The `__growTable()` method in [Listing 11-5](#) first sets `oldTable` to reference the current hash table and estimates the `size` of the next table to be twice the old size, plus one. Then it starts a loop that finds the first prime number that equals or exceeds that size. Why? That's because prime numbers have special importance with algorithms that use the modulo operator. When you choose a prime number for the size, only multiples of that prime number hash to cell 0. Similarly, only multiples of that prime number plus one hash to cell 1. If the keys to be inserted in the hash table do not have that prime number as a factor, they tend to hash over the whole range of cells. That's very desirable behavior, as you will see later.

The test for prime numbers, `is_prime()`, is not a part of standard Python. There are many published algorithms for this (deceptively simple) test, so we don't show it here.

Rehashing

After deciding the new `size` of the hash table, the `__growTable()` method in [Listing 11-5](#) creates the new array and sets the number of items back to zero. That might seem odd; why would we want an empty hash table at this point? The reason is that the key-value pairs in `oldTable` need to be stored in the new table, but in new positions, and none of them are in place yet. If you simply copy the contents of a cell in `oldTable` to the cell with the same index in the new array, the `__find()` method might not find it. The new array size affects where the algorithm starts its search because it is used in the modulo operation that computes the hash address. For example in [Figure 11-12](#), the linear probe for key 6,378 started at cell 78 and eventually found the item in cell 81 due to collisions. That was when the array size was 100. If the array size is 200, the linear probe would start at cell 178 (6,378 modulo 200). Storing that item at cell 78 in the new table would work only if there were a large cluster extending from 178 through cell 199 and then wrapping around from 0 to 78.

Instead of copying, *key-value pairs must be reinserted*, a process called **rehashing**, to ensure proper placement. The insertion process distributes them to their new cells, perhaps causing collisions, but probably fewer collisions than occurred in the smaller array. The `__growTable()` method in [Listing 11-5](#) loops over all the cells in the smaller array and reinserts any filled cells that are not simply the deleted cell marker. This operation can be quite time-consuming, and it must scan all the cells using the `range(len(oldTable))` iterator, not just the `__nItems` known to be filled.

One implementation note: the asterisk in the `self.insert(*oldTable[i])` expression tells Python to take the tuple stored at `oldTable[i]` and use its two components as the two arguments in the call to `insert()`, which are `key` and `value`. The asterisk (*) means multiplication in most contexts but has a different meaning when it precedes the arguments of a function call or elements of a tuple.

The `linearProbe()` Generator

Let's return to the part of the `insert` process that probes for empty cells. The `__find()` method in [Listing 11-4](#) has a loop of the form

```
for i in self.__probe(self.hash(key), key, self.cells()):
```

This is the place where the `__probe` attribute of the object is called to generate the sequence of indices to check. The default value for the `__probe` attribute is `linearProbe()`, which is shown in [Listing 11-6](#).

Listing 11-6 *The `linearProbe()` Generator for Open Addressing*

```
def linearProbe(                # Generator to probe linearly from a
    start, key, size):          # starting cell through all other cells
    for i in range(size):       # Loop over all possible increments
        yield (start + i) % size # and wrap around after end of table
```

The `linearProbe()` is a straightforward generator that behaves similarly to Python's `range()` generator. In fact, it uses `range()` in an internal loop that steps a variable, `i`, through all `size` cells of the table. The `i` index is added to the starting index for the probe, so it will examine all the subsequent cells in the array. When that offset index goes past the end of the table, the iterator wraps the index back to zero by using the modulo of the offset index with `size`. The new index will always be between zero and one less than `size`.

The `yield` statement in the loop body sends the new index back to the `__find()` method to be checked ([Listing 11-4](#)). Remember that the `yield` statement returns a value and control to the caller. The caller then uses the value in its own loop until it's time to get the next value from the iterator. Control then passes back to the iterator right after the `yield` statement. In this case, `linearProbe()` goes on and increments its own `i` variable.

When `linearProbe()` finishes going through all the indices of the array, the generator ends (by raising a `StopIteration` exception). That signals to the caller, `__find()`, that all the cells have been probed. If the `insert()` method hasn't found an empty cell before the linear probe sequence finishes, then the table must be full.

The delete() Method

Deleting items is straightforward for hash tables because you only need to mark the deleted cells. Like with insertion, the `delete()` method starts by using the `__find()` method to find the cell containing the item to delete, as shown in [Listing 11-7](#). After the cell index, `i`, is determined, the behavior depends on what is stored at the cell. If `__find()` could not discover that cell, or it already contains a deleted element or some other item whose key does not match, then a possible error has been found. This `delete()` method has an optional parameter, `ignoreMissing`, which determines whether an exception should be raised. In general, data structures that store and retrieve data should raise exceptions when the caller tries to remove an item not in the store, but in some circumstances such errors can be safely ignored.

Listing 11-7 *The delete() Method of HashTable*

```
class HashTable(object):    # A hash table using open addressing
...
    def delete(self,        # Delete an item identified by its key
               key,        # from the hash table. Raise an exception
               ignoreMissing=False): # if not ignoring missing keys
        i = self.__find(key) # Look for cell index matching key
        if (i is None or    # If the probe sequence fails or
            self.__table[i] is None or # cell i is empty or
            self.__table[i][0] != key): # it's not the item to delete,
            if ignoreMissing: # then item was not found. Ignore it
                return        # if so directed
            raise Exception(  # Otherwise raise an exception
                'Cannot delete key {} not found in hash table'.format(key))

        self.__table[i] = HashTable.__Deleted # Mark table cell deleted
        self.__nItems -= 1    # Reduce count of items
```

When the `delete()` method finds a cell with a matching key, it marks the cell with the special `__Deleted` marker defined for the class and decrements the

count of the number of stored items. Typically, no attempt to resize the hash table is made when many deletions cause the load factor to shrink below the threshold used to determine when to grow the table. That's based on the assumption that deletion will occur much less often than insertion and search and the cost of having to rehash all the items stored in the table.

The `traverse()` Method

To traverse all the items in a hash table, all the table cells must be visited to determine which ones are filled. The process is easy to implement as a generator. The one special consideration is that deleted items should not be yielded. [Listing 11-8](#) shows the implementation.

Listing 11-8 *The `traverse()` Method of `HashTable`*

```
class HashTable(object):    # A hash table using open addressing
...
    def traverse(self):      # Traverse the key, value pairs in table
        for i in range(len(self.__table)): # Loop through all cells
            if (self.__table[i] and # For those that contain undeleted
                self.__table[i] is not HashTable.__Deleted): # items
                yield self.__table[i] # yield them to caller
```

Because the implementation stores the key-value pairs as (immutable) Python tuples, they can be yielded directly to the caller, which can assign them to two variables in a loop such as

```
for key, value in hashTable.traverse():
```

Alternatively, callers can use a single loop variable holding the pair as a tuple.

The Traverse and New Buttons

Returning to the visualization tool, the Traverse button launches the preceding loop. Each item's key is printed in a box (ignoring its data). It illustrates the `traverse()` iterator skipping over empty and deleted cells.

You can create new, empty hash tables with the New button. This button takes two arguments: the number of initial cells and the maximum load factor. You can specify starting sizes of 1 to 61 cells and maximum load factors from 0.2 to

1. When invalid arguments are provided, the default values of 2 and 0.5 are used.

If you create hash tables with nonprime sizes, they will grow using the `__growTable()` method of [Listing 11-5](#), setting the new size to a prime number. Try stepping through the animation of the rehashing process. This animation shows how the items move to their new cells.

If you want to see the effects of using different table sizes, try using the New button to create a table of the desired size with a maximum load factor of 0.99. The table will not grow until it becomes completely full, so you can see the effects of different table sizes and clustering.

Quadratic Probing

Using open addressing, hash tables can find empty (and deleted) cells to fill with new values, but clusters can form. As clusters grow in size, it becomes more likely that new items will hash to cells within a cluster. The linear probe steps through the cluster and adds the new item to the end, making it even bigger, perhaps joining two clusters.

This behavior is somewhat like that of automobiles entering a highway. If only isolated vehicles make up the flow of highway traffic, the arriving vehicles have plenty of gaps to fit into. When the highway is crowded, longer chains of vehicles form clusters. Newly arriving vehicles wait for the cluster to pass and join at the end, increasing the cluster size. Hopefully, the arriving vehicles don't cause real-world collisions as they "probe" for an open spot on the highway.

The likelihood of forming clusters and the size of clusters depend on the ratio of the number of items in the hash table to its size—its load factor. Clusters can form even when the load factor isn't high, especially when the hashing function doesn't distribute keys uniformly over the table. Parts of the hash table may consist of big clusters, whereas others are sparsely populated. Clusters reduce performance.

Quadratic probing is an attempt to keep clusters from forming. The idea is to probe more widely separated cells instead of those adjacent to the primary hash site.

The Step Is the Square of the Step Number

In a linear probe, if the primary hash index is x , subsequent probes go to $x + 1$, $x + 2$, $x + 3$, and so on. In quadratic probing, subsequent probes go to $x + 1$, $x + 4$, $x + 9$, $x + 16$, $x + 25$, and so on. The distance from the initial probe is the square of the step number: $x + 1^2$, $x + 2^2$, $x + 3^2$, $x + 4^2$, $x + 5^2$, and so on.

Figure 11-13 shows some quadratic probes.

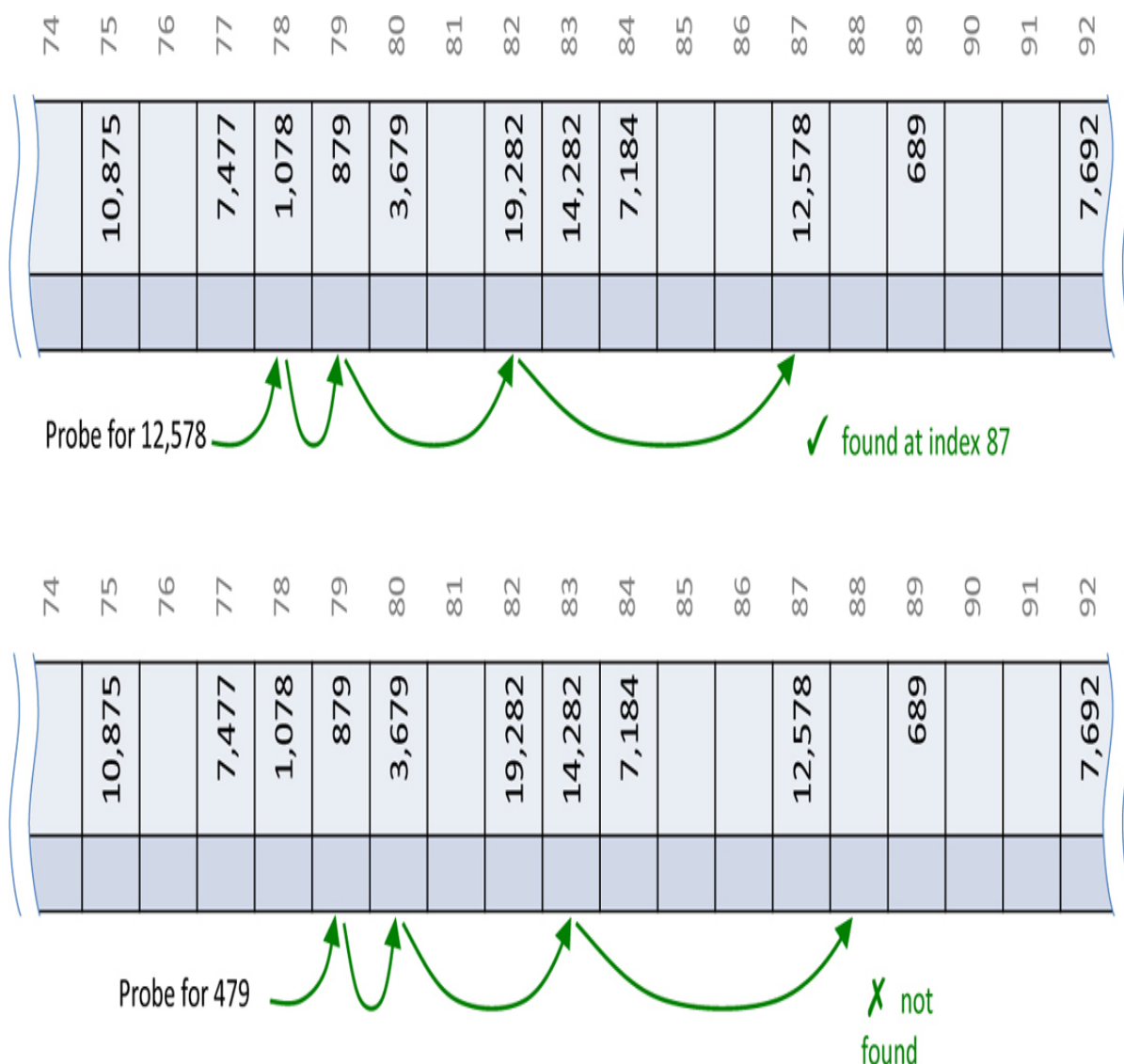


Figure 11-13 Quadratic probes

The quadratic probe starts the same as the linear probe. When the initial probe lands on a nonmatching key, it picks the adjacent cell. If that's occupied, it may be in a small cluster, so it tries something 4 cells away. If that's occupied, the cluster could be a little larger, so it tries 9 cells away. If that's occupied, it really starts making long strides and jumps 16 cells away. Pretty soon, it will

go past the length of the array, although it always wraps around because of the modulo operator.

Using Quadratic Probing in the Open Addressing Visualization Tool

The HashTableOpenAddressing Visualization tool can demonstrate different kinds of collision handling—linear probing, quadratic probing, and double hashing. (We look at double hashing in the next section.) You can choose the probe method whenever the table is empty by selecting one of the three radio buttons. When the table has one or more items, the buttons are disabled to preserve the integrity of the data.

To see quadratic probing in action, try the following. Use the New button to create a hash table of 21 cells with a maximum load factor of 0.9. Select the Use quadraticProbe button to switch to quadratic probing. Then use the Random Fill button to insert 12 random keys in the table. This action produces a table with various filled sequences, like the one shown in [Figure 11-14](#). There is a 6-cell filled sequence along with several shorter ones.

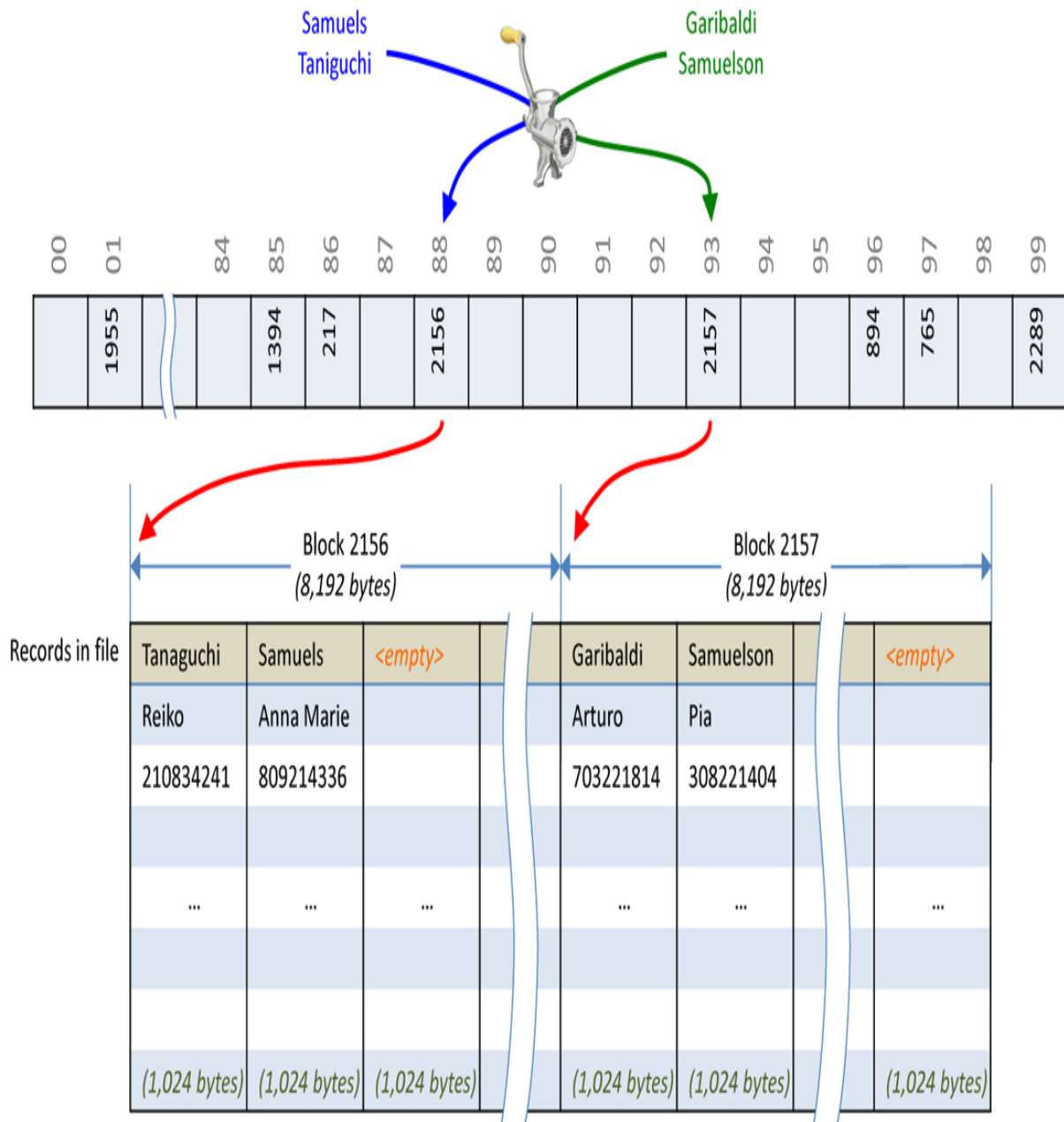


Figure 11-14 *Quadratic probing in the HashTableOpenAddressing Visualization tool*

In this example, try inserting the key 0. Because it is a numeric key, the hashed value is 0 and the first probe is to cell 0. After finding cell 0 full, it tries the cell at $0 + 1$. That cell is occupied, so it continues to cell $0 + 4$, finding another stored item. When it reaches $0 + 9$, it finds cell 9 empty and can insert key 0 there. It's easy to see how the quadratic probes spread out further and further.

If you next try to insert the key 21, it will hash to cell 0 for its initial probe again because the table has 21 cells. The insertion now will repeat the same set of probes as for key 0 and then continue on to locate an empty cell. Perhaps surprisingly, it revisits some of the same cells during the probing, as shown in [Figure 11-15](#).



Figure 11-15 *Inserting key 21 by quadratically probing a relatively full hash table*

Specifically, the cells probed to insert key 21 are 0, 1, 4, 9, 16, (25) 4, (36) 15, (49) 7. The indices in parentheses are the values before taking the modulo with 21.

This example is particularly troublesome. Not only does it have to probe all the cells probed to insert key 1, but it also repeats the probe at cell 4 that wouldn't have occurred using linear probing. If you keep inserting more keys, you will see how this behavior becomes worse when the table is almost full. With the max load factor set to 0.9, it won't grow the table until the 19th key is inserted.

Incidentally, if you try to fill the hash table up to the maximum 61 items it supports or with a very high maximum load factor, the visualization tool may not be able to insert an item, even if empty cells remain. The program tries only 61 probes before giving up (or whatever the size of current table is). Because quadratic probes can revisit the same cells, the sequence may never land on one of the few remaining empty cells.

Also try some searches when the table is nearly full, both for existing keys and ones not in the table. The probe sequences get very long in some cases.

Implementing the quadratic probe is straightforward. Listing 11-9 shows the `quadraticProbe()` generator. Like the `linearProbe()` shown in Listing 11-6, it uses `range()` to loop over all possible cells one time. The loop variable, `i`, is squared, added to the `start` index, and then mapped to the possible indices using the modulo operator. Because `i` starts at zero, the first index that is yielded is the `start` index.

Listing 11-9 *The `quadraticProbe()` Generator for Open Addressing*

```
def quadraticProbe(          # Generator to probe quadratically from a
    start, key, size):       # starting cell through all other cells
    for i in range(size):    # Loop over all possible cells
        yield (start + i ** 2) % size # Use quadratic increments
```

The Problem with Quadratic Probes

Quadratic probes reduce the clustering problem with the linear probe, which is called **primary clustering**. Quadratic probing, however, suffers from different and more subtle problems. These problems occur because all the probing sequences follow the same pattern in trying to find an available cell.

Let's say 184, 302, 420, and 544 all hash to address 7 and are inserted in this order. Then 302 will require a one-cell offset, 420 will require a four-cell offset, and 544 will require a nine-cell offset from the first probe. Each additional item with a key that hashes to 7 will require longer probes. Although the cells are not adjacent in the hash table, they still are causing collisions. This phenomenon is called **secondary clustering**.

Secondary clustering is not a serious problem. It occurs for any hashing function that places many keys at the same initial cell or at multiple cells where the fixed pattern for finding empty cells overlap. There's another issue, however, and that's the coverage of cells visited by the probing sequence.

The quadratic probe keeps making larger and larger steps. There's an unexpected interaction between those steps and the modulo operator used to map the index onto the available cells. In the linear probe, the index is always incremented by one. That means that linear probing will eventually visit every cell in the hash table after wrapping around past the last index.

In quadratic probing, the increasing step sizes mean that it eventually visits only about half the cells. The example in [Figure 11-15](#) illustrated part of the problem when it revisited cell 4. The reason for this behavior takes some mathematics to explain.

If you look at the spacing between the cells probed, you'll see that it increases by two at each step. The spacing between $x + 1$ and $x + 4$ is three. The spacing between $x + 4$ and $x + 9$ is five. The spacing between $x + 9$ and $x + 16$ is seven, and so on. It already looks as though it might skip every other cell because it will stay on the odd cells if the initial probe was to an even cell (and vice versa). That's actually not the case because the modulo operator will change between odd and even numbered cells when the index goes past the modulo value. That value is usually a prime number, so it is odd.

Even with a prime number of cells, however, the quadratic probe starts repeating the same sequence of cell indices fairly quickly. Here's a simple example. For simplicity, let's say the hash table has seven cells in it, and the key to store initially hashes to cell index 0. Quadratic probing then visits indices 1, 4, 9, 16, 25, 36, 49, 64, and so on. After taking the modulo with seven, however, the full sequence is 0, 1, 4, 2, 2, 4, 1, 0, 1, 4, 2, 2, 4, 1, 0, and so on. That 0, 1, 4, 2, 2, 4, 1 sequence repeats forever, leaving out cell indices 3, 5, and 6.

Three cells may not seem like much, but they're three out of seven cells total. Even worse, the probe revisits indices 1, 2 and 4 twice. Because they've already been visited during the seven probe sequence, they must already be occupied, so revisiting them just wastes time (much more time than the single cell revisited in the example in [Figure 11-15](#)). As the prime number of cells gets bigger, the repetitive behavior continues. After the quadratic term grows to be the square of the table size, the sequence returns to the starting index. Eventually, about half of the cells are visited, and half are not.

So linear probing ends up causing primary clustering, whereas quadratic probing ends up with secondary clustering and only half the coverage of the hash table. This approach is not used because there's a much better solution.

Double Hashing

To eliminate secondary clustering as well as primary clustering and to help with hash table coverage, there's another approach: **double hashing**.